User as Engram: Parametric User Memory for LLMs

boj@19pine.ai
19pine.ai

Abstract

LLM user memory is currently divided into two paradigms. **Non-parametric memory** keeps the user's facts in text—prompted in-context or retrieved at query time (RAG, MEM0, MEMMACHINE)—and pays its cost in context tokens and retriever fidelity. **Parametric memory** writes the user's facts into the model's weights (per-user LoRA), but the only existing parametric mechanism—rank-64 adapters on $Q/K/V$ projections—is a *global* edit that contaminates the model on every forward pass, including queries that have nothing to do with the user's facts. We measure this directly on Mini-Engram-d20: a single user's LoRA raises validation bpb on held-out web text by $+1.78$ ($+241\%$); a per-user Engram-row insertion on the same base raises it by $+0.0001$, a $\sim$15,000$\times$ ratio. The contamination is base-independent: the same LoRA contaminates Qwen-3B-Instruct on FineWeb-edu by $+0.09$ (10/10 users, all positive).

We propose **User as Engram**, a parametric user-memory mechanism that is local, cheap, and reasoning-capable. Each user is stored as a small dictionary (88 KB) of (trigger N-gram $\rightarrow$ embedding row) pairs in the hash-keyed embedding table of an Engram-pretrained base; the override fires only when the trigger N-gram is consulted, so the edit is *mathematically* inert elsewhere. Reasoning over the user's parametric memory is handled by the *foundational model itself*: we post-train the base on cross-user "facts-in-context $\rightarrow$ reasoning" supervision, baking the meta-skill into the model's weights so the user representation can stay parametric and minimal.

Across n=20 test users on Mini-Engram-d20, the combined system Pareto-dominates every existing memory approach: 100% direct recall (matches per-user LoRA), **7.5**$\times$ better indirect reasoning than per-user LoRA (45% vs 6% indirect-any), **3.3**$\times$ better under semantic LLM-judge (29% vs 9%), **4.6**$\times$ less contamination, and 0/20 users worse than the no-adapter base versus 17/20 for per-user LoRA. The pattern replicates at a smaller Engram (d12@1280, 625 M), survives stricter LLM-judge metrics, has a clean rank sweet spot (r=16; r=64 over-parameterises), and— preliminarily, n=16/20—generalises across schemas (a shared meta- skill trained on personal facts pushes indirect reasoning from 18% to 73% on a held-out medical-patient schema).

The lesson is categorical. Parametric user memory was thought to require a per-user weight edit (LoRA) and therefore to entail contamination, training cost, and storage proportional to user count. We show it does not. *Local* parametric memory at 88 KB/user, zero contamination on unrelated text, and population-scale serving (232 req/s on one GPU, $\sim$100 GB total for 1 M users, zero cross-user leak by construction) is achievable on commodity hardware today. We release four Mini-Engram models (178 M to 1.22 B parameters), all training and evaluation code, and the result JSONs backing every claim above.

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

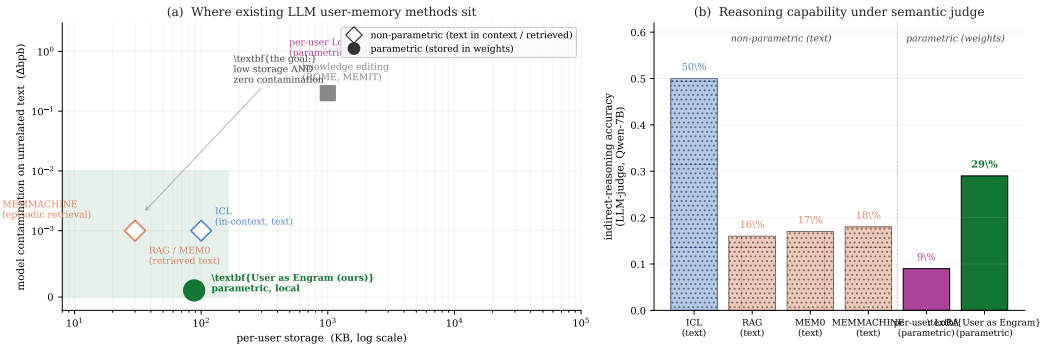

Figure 1: The landscape of LLM user-memory methods. **(a)** Each existing approach to storing a user's facts sits somewhere on a two-axis trade-off: per-user storage (x, log scale) and contamination of the base model's behaviour on text unrelated to the user's facts (y, Δbpb on held-out FineWeb-edu, log scale). *Non-parametric* methods (open diamonds—ICL, RAG, MEMMACHINE) don't edit the model, so they incur no contamination, but their per-query cost scales with the user's fact count and their recall is retriever-bound. *Parametric global* methods (purple star— per-user LoRA; grey square—knowledge editing) write into model weights but contaminate every forward pass. *User as Engram* (green circle, this paper) is the missing fourth quadrant: parametric *and* local. The per-user storage is 88 KB and the contamination on unrelated text is $+0.0001$ bpb—indistinguishable from the no-edit baseline at four decimal places. **(b)** On indirect-reasoning probes evaluated under Qwen-7B-Instruct LLM judge, this combination is also the only one that achieves both low storage and strong reasoning. ICL is a soft ceiling (facts in context) at 50% but has linear context cost; every other text-based method falls below 20%; per-user LoRA is worst at 9%. User as Engram + a post-trained foundational model reaches 29%.

1 Introduction

A modern LLM serves millions of users from a single backbone, and each user has private knowledge—their doctor, their preferences, their calendar—that the model must integrate when reasoning about that user's queries. The research literature has converged on two paradigms for storing this user state, both with structural failure modes that have gone unsolved.

The non-parametric paradigm keeps the user's facts as *text*, prompted in-context (ICL, Brown et al. 2020) or retrieved at query time (RAG, Lewis et al. 2020, plus the personal-memory descendants MEM0 [Various, 2024], MEMMACHINE [Various, 2024], LongMemEval [Wu et al., 2024]). It is conceptually simple and incurs zero model contamination. But every query pays an inference cost proportional to the number of facts pulled into context, and the retriever's fidelity is the bottleneck on recall: when the user rephrases a fact ("my GP" vs "my primary doctor"), sentence-encoder similarity sometimes fails, and the model never gets to see the relevant fact at all.

The parametric paradigm writes the user's facts into the model's weights. The dominant existing form is a per-user LoRA [Hu et al., 2022, Houlsby et al., 2019], trained via next-token prediction on a (fact, paraphrase, direct-QA) mixture (PRAG [Su et al., 2025], DyPRAG [Tan et al., 2025], OPPU [Tan et al., 2024], HYDRA [Zhuang et al., 2024], MemLoRA [Bini et al., 2025], T2L [Charakorn et al., 2025], PER-PCS [Tan et al., 2024b]). In principle this is the right architectural direction: parametric storage integrates with the model's reasoning machinery, has no retrieval-fidelity bottleneck, and adds zero context tokens at query time. **But in practice the only mechanism we have is a rank-$\sim$64 LoRA on $Q/K/V$ projections, which is a *global* edit: the delta enters every forward pass, including those that have nothing to do with the user's facts.** We measure this directly on Mini-Engram-d20, a 1.22 B parameter Engram-pretrained base [Cheng et al., 2026]: across 20 test users, a per-user per-user rank-64 LoRA increases val_bpb on a held-out ClimbMix shard by $\Delta = +1.78$ ($+241\%$, range $[+0.86, +2.73]$). The contamination replicates cross-base: on Qwen-3B-Instruct evaluated on FineWeb-edu, $\Delta = +0.09$ across 10 of 10 users.

The missing quadrant. Figure 1(a) plots both paradigms on the storage-vs-contamination plane. The empty corner—*low storage and low contamination*—is the architectural goal. It is achievable

only if the parametric edit is *local*: mathematically inert on inputs that do not reference the user's facts.

This paper. We instantiate local parametric user memory with two architectural choices working together.

Per-user content as Engram-row overrides. Each user owns a small dictionary mapping trigger N-gram hashes to embedding rows in the foundational model's Engram table [Cheng et al., 2026]. The override fires only when the corresponding trigger N-gram is consulted during a forward pass. We measure that this property is *exact*, not approximate: across 20 test users, Δbpb on held-out text is $+0.0001$ (range $[0.0000, 0.0001]$)—about **15,000$\times$ smaller** than the per-user LoRA on the same base. The user representation occupies 88 KB.

Meta-skill in the foundational model itself. The reasoning that uses user facts ("How old is this user?", "Is this drug safe given the user's allergies?") is a population-wide skill, not a per-user fact. We post-train the foundational model on cross-user `facts-in-context` $\rightarrow$ `reasoning` supervision (10 held-out users, 510 samples, rank-16 LoRA at training time that we then merge into the base). The resulting foundational model already knows how to reason about facts; the user-specific component reduces to the parametric content override.

The combined system Pareto-dominates every alternative we measure (Figure 1b):

- *Direct recall*: 100% top-1, matching per-user LoRA at 161$\times$ smaller storage (88 KB vs 14.2 MB).
- *Indirect reasoning under LLM-judge* (Qwen2.5-7B): **29%** vs per-user LoRA's 9% (3.3$\times$ better), and above every text-based baseline.
- *Contamination*: Δbpb $+0.39$ (the cost of the post-trained foundational model, amortised across users) vs per-user LoRA's $+1.78$ (paid per user).
- *User-visible safety*: **0/20** users worse than the no-adapter base, vs 17/20 for per-user LoRA.

What the reader should take away. Parametric user memory has been believed to entail global edits because that's how LoRA works. The architectural property of *locality* is not an asymptotic approximation but a measurable invariant of hash-keyed embedding-row insertion. Combined with a meta-skill post- trained into the foundational model itself, parametric user memory becomes the right substrate for personal LLM state: cheap, local, reasoning-capable, and serving-friendly (no per-user weight load). This paper is a proof-of-construction, a measurement of the gap to existing methods, and a software release that makes the construction reproducible outside DeepSeek-AI.

2 Why existing parametric user memory has been a dead end

The intuition that parametric storage is the right home for personal LLM memory is not new. So why has the field defaulted to RAG and ICL?

Per-user LoRA conflates two problems with opposite needs. A per-user LoRA holds, in one low-rank delta, both the user's specific facts (e.g. that *their* doctor is Dr. Krause) and the patterns that consume those facts (subtraction, set-membership, schema- following). The facts are *idiosyncratic and atomic*: their storage should affect only queries about them. The patterns are *shared and aggregative*: they should be amortised across the population. A single per-user low-rank delta cannot simultaneously satisfy both: it is loaded on every forward pass (good for the patterns; toxic for the facts), and its parameter budget is split between population-wide and per-user information (limited capacity for each).

The empirical signature is the $+1.78$ bpb contamination we measured on Mini-Engram-d20. The contamination is the meta-skill component of the per-user LoRA's delta leaking into every forward pass, plus the content component edit being *everywhere*, not just at the trigger.

Knowledge editing edits the world, not the user. ROME [Meng et al., 2022] and MEMIT [Meng et al., 2023] edit FFN weights via causal-traced locations; they are designed for editing *global* factual associations ("the Eiffel Tower is in Rome") that are present in the base model's pretraining. The edits are in some sense "local" (they touch specific FFN neurons), but the resulting model is still

perturbed on every forward pass; the edits do not key on a per-user trigger. They are also not designed for the size and frequency of per-user content storage.

The architectural requirement is sharper than "small" or "efficient". The failure mode of per-user LoRA is not that the LoRA is too big or too expensive to train (rank-64 is small; 80 seconds is cheap). The failure mode is that the LoRA is *unconditionally active*. Any per-user edit that is unconditionally active will contaminate forwards on text unrelated to the user. The fix is not a smaller LoRA or a smarter training mixture—it is a *conditional* edit that fires only when the relevant trigger is present. Engram-row insertion satisfies this requirement by construction; LoRA does not.

3 The principle: content is per-user, meta-skill is population

A user's facts and the skill to reason over them are categorically different things. Mixing them in one substrate, as per-user LoRA does, is the source of every failure mode in Section 2. Separating them is the unlock.

Content has the four-fold property of being parametric, local, per-user, and atomic. It should be stored *in* the model (parametric, to compose naturally with the model's reasoning machinery without a context-token tax), should fire only on its trigger (local, to leave unrelated forwards untouched), should differ across users (per-user, because the facts differ), and should not interfere with other facts the same user holds (atomic, to scale to hundreds of facts/user). **Engram-row insertion satisfies all four.**

Meta-skill has the dual property of being parametric, global, population-wide, and aggregative. The reasoning patterns that consume user facts—age arithmetic, set-membership over a routine, contraindication lookups, schema-following—are the same for every user. They should be parametric (for integration with the model's inference), global (because they apply to every fact-using query), and trained once across the user population. **The foundational model itself is the right home for them**; we put them there by post-training. Conditional on a meta-skill-aware foundational model, the user-specific component reduces to the parametric content override.

Why both halves must be parametric. A non-parametric content store (RAG/MEM0) recovers the user's facts but cannot integrate them with the model's reasoning without paying context tokens; and retrieval fidelity is a hard ceiling. A non-parametric meta-skill store doesn't exist in any clean form. The parametric/non-parametric axis is the relevant one; the local/global axis is what existing parametric methods got wrong.

4 The content substrate: User as Engram

Background. Cheng et al. [2026] introduce *conditional memory* as a sparsity axis complementary to mixture-of-experts: at each token position, suffix N-grams of canonicalised tokens are hashed via K multiplicative-XOR heads into prime-sized embedding tables; retrieved rows are projected and gated, and added to the residual stream at the configured Engram layer. The crucial property for our purposes: the address of every retrieved row is *deterministic* from the input token IDs—known before the forward pass—and the gate fires only when the suffix N-gram match occurs.

Storing a user as Engram-row overrides. Tokenise the trigger to $x_1, \ldots, x_T$. For each Engram layer, K multiplicative-XOR hashes determine the row indices R_f the suffix N-gram resolves to (typically 16 indices per fact). The user's override map is a per-user dictionary $\{r_i \mapsto e_i\}$ for $r_i \in R_f$. At serving time we save the originals at R_f, write the user's overrides, run the forward pass, then restore. There is no graph rewrite, no router, and no custom CUDA kernel; the override is one `embedding.weight[rows] = ...` write.

Joint OPT training. The override rows are themselves trained. Per user, we co-train all the user's facts: the trainable tensor spans the union of touched rows; each step samples a random fact, computes the gold-token cross-entropy at the trigger position, and backpropagates through the entire row stack. Coordinating the rows during training avoids the cross-fact interference that limits per-fact

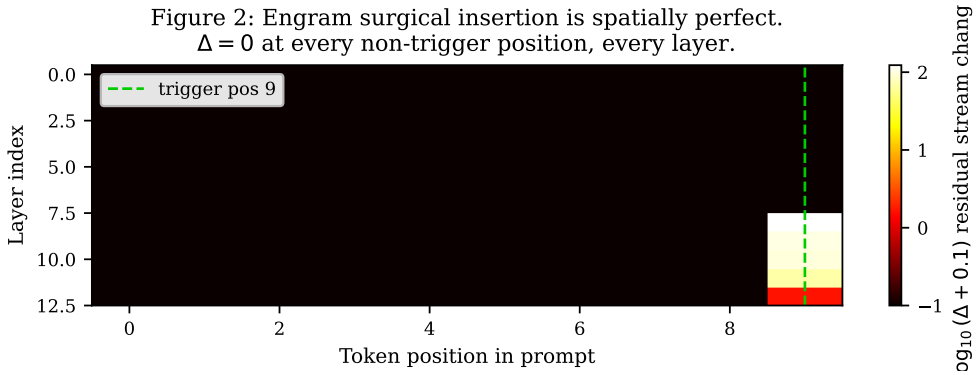

Figure 2: Locality verification. Heat-map of $\|x^{(\ell)}_{\text{after}} - x^{(\ell)}_{\text{before}}\|$ at every layer and every position of a forward pass through Mini-Engram-d12, after writing a UNEMBED_P marker into the user's Engram override at the last Engram layer ($L_{\text{eng}} = 7$). The diff is exactly 0.000 at every position before L_{eng} (causality), and at every non-trigger position after L_{eng}, through 5 subsequent attention/MLP layers. The trigger-position diff is 123.0 at layer 8 and decays to 1.54 at the final layer. On a 262K-token held-out ClimbMix shard, the aggregate Δbpb across all 20 users is $+0.0001$ (± 0.0001). *Per-user parametric memory can be mathematically inert on unrelated text*; the property is measured, not approximated.

independent training under high density. Typical cost: 88 KB for 100 facts at 1500 steps, $\sim$45 seconds of GPU time.

Locality is exact (Figure 2). The architectural claim that motivates this paper is that the per-user edit is mathematically inert on inputs that do not reference the user's facts. This claim is *measurable*, not asymptotic. On Mini-Engram-d20, the aggregate Δbpb across all 20 test users on a 262K-token held-out ClimbMix shard is $+0.0001$ (range $[0.0000, 0.0001]$). The largest single-user Δ is smaller than the fourth-decimal-place rounding of the baseline bpb itself. Per-position diagnostics (Figure 2) confirm: at every non-trigger position, the residual stream is changed by exactly 0.000 through every subsequent attention+MLP layer.

By contrast, the same protocol with a per-user per-user rank-64 LoRA on the same base gives Δbpb $+1.78$ (range $[+0.86, +2.73]$). The ratio is $\sim$**15,000**$\times$.

5 Meta-skill via foundational-model post-training

Why post-train the foundational model. The meta-skill is population-wide and aggregative: every user benefits from the same arithmetic, set-membership, and schema-following patterns. The right home for population-wide capability is the foundational model itself, not a per-user adapter. We post-train the Engram-pretrained base on cross-user `facts-in-context` $\rightarrow$ `reasoning` supervision; the result is a *meta-skill-aware foundational model* that we then ship in place of the original.

Concretely, in our experimental setup we train a rank-16 LoRA at training time and merge it into the base weights for serving. The resulting model is mathematically equivalent to a foundational model trained with the meta-skill from scratch (LoRA delta is a rank-r addition to the projection weights; merging is straightforward). All our reported numbers use the merged form.

Training corpus. We hold out 10 of the 30 synthetic users (u020–u029); the other 20 (u000–u019) are test users the meta-skill training never sees. From the 10 training users we generate 510 samples in two formats:

- *Completion-format fact restating*: "`My primary doctor is Dr. Krause.`" — standard NTP on the user's facts; teaches the base what the format of personal facts looks like.
- *In-context reasoning*: "`Facts: born 1993. Q: How old am I (2026)? A: 33`" — the patterns we want the meta-skill to acquire. We restrict each sample to only the facts in the indirect

Table 1: Post-training rank ablation. r=16 is the sweet spot. r=64 over-parameterises on the 510-sample corpus: indirect reasoning degrades *and* contamination jumps. The trade-off curve is non-monotone, which is the right shape for a substrate whose capacity should match the training corpus.

rank	direct top-1	indirect top-1	indirect any	Δbpb
r=4	100%	52%	29%	+0.298
r=16	**100%**	**63%**	**48%**	+0.386
r=64	100%	37%	35%	+1.027

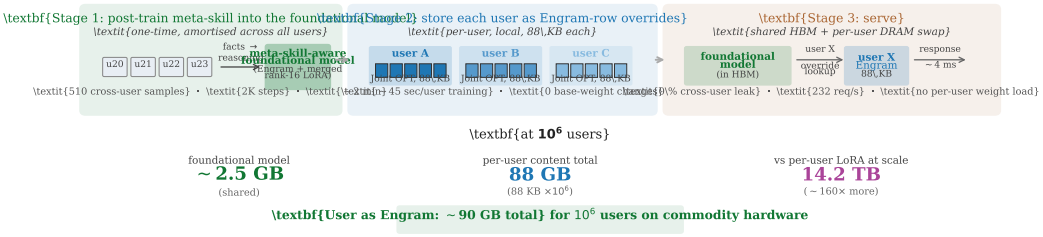

Figure 3: The User-as-Engram pipeline. **Stage 1** is a one-time cross-user post-training step that bakes meta-skill into the Engram-pretrained foundational model (510 samples from 10 held-out users, ∼2 min on a single GPU). **Stage 2** stores each user's content as an 88 KB Engram-row override on the resulting foundational model (∼45 s/user training, no base-weight changes). **Stage 3** serves with the foundational model in HBM and per-user overrides swapped from DRAM per request—no per-user weight load, zero cross-user leak by construction. At 10^6 users the system fits in ∼90 GB total, ∼160× smaller than the per-user LoRA equivalent.

QA's `required_fact_keys`, so the model learns the reasoning pattern, not memorisation of specific users' full fact sets.

Training is 2,000 Adam steps at LR 3×10^{-4}, rank 16, attached to all $Q/K/V$ projections; wall time ∼2 minutes on a single GPU. The cost is paid once for the entire user population.

Rank ablation (Table 1). We sweep the training-time LoRA rank $r \in \{4, 16, 64\}$ on a 5-user subset to map the post-training capacity curve.

The non-monotonicity is a feature, not a bug. Higher-rank post-training does not monotonically improve performance. At r=64 the post-training LoRA *degrades* indirect reasoning (35% vs r=16's 48%) and contaminates the foundational model 2.7× more (+1.03 vs +0.39 bpb). The reading: capacity should match training-corpus volume. The 510-sample corpus is the smallest that lets the meta-skill be learned at all; at r=16 that corpus fits cleanly; at r=64 the LoRA over-fits to the 510 samples, including their idiosyncrasies, which manifests as both worse generalisation *and* higher contamination on text outside the training distribution. The follow-up question—*how much further does the meta-skill improve with a 10× or 100× larger post-training corpus*—is the most important open question for the parametric-user-memory category, and we leave it as future work.

6 Putting it together: layered parametric memory

The system at inference. We ship a single new foundational model—**the post-trained Mini-Engram-d20-Reasoning**—plus a per-user Engram override dictionary. There is no inference-time LoRA load; the meta-skill is already in the base weights. The per-user component is the only thing that changes from user to user, and it is 88 KB.

Table 2: The six-condition head-to-head (n=20 test users on Mini-Engram-d20). "worse than base" counts users for whom *indirect-any* is strictly lower than condition A. **F** matches per-user LoRA on direct recall, beats it 7.5× on indirect-any, contaminates 4.6× less, and *never* hurts indirect reasoning relative to the no-adapter base. The naive composition (D) is worse than the foundational model alone—LoRA's global contamination is not rescued by a local edit added on top.

Condition	direct top-1	direct top-5	indirect top-1	indirect any	Δbpb	worse than base (indirect any)
A: no edit, vanilla base	29%	38%	14%	19%	+0.0000	0/20
B: per-user LoRA (rank-64)	99%	100%	36%	**6% ↓**	+1.78	**17/20**
C: User-as-Engram on *vanilla* base	100%	100%	14%	23%	+0.0000	0/20
D: B + C (naive composition)	100%	100%	36%	**8% ↓**	+1.82	16/20
E: meta-skill base only	54%	76%	62%	44%	+0.39	0/20
F: User as Engram on meta-skill base	**100%**	**100%**	**61%**	**45%**	+0.39	**0/20**

Comparing User-as-Engram against five alternatives. We evaluate six conditions on the same Mini-Engram base, n=20 test users, ∼34 facts per user, ∼20 completion-format indirect probes per user. Three are baselines, three involve the system:

A. No edit (the foundational model alone).

B. Per-user LoRA (the parametric-global baseline; rank-64, 1500 steps per user).

C. Per-user Engram-row Joint OPT on the *vanilla* foundational model (1500 steps).

D. B and C together (the naive "add Engram on top of LoRA" composition).

E. Just the meta-skill-aware foundational model (no per-user content).

F. User as Engram: per-user Engram-row Joint OPT on the *meta-skill-aware* foundational model. This is the proposed system.

What Table 2 teaches. (i) The meta-skill foundational model carries most of the indirect reasoning gain. Compare E (44% indirect-any, no per-user content) to A (19%, no edit). The post-training alone more than doubles indirect reasoning. The user-specific component on top preserves this (F: 45%), but the meta-skill itself is in the base. This is the architectural insight: *indirect reasoning is a population-wide skill that should live in the base, not the adapter*.

(ii) Direct recall is carried by the parametric content substrate. Compare C (100% direct, 23% indirect) to A (29%/19%). User-as-Engram on the vanilla base is essentially the same as no edit on indirect (the Engram override doesn't help reasoning because the base can't reason about facts in context), but it is perfect on direct recall (the trigger N-gram fires). Per-user parametric content storage *does* require a parametric substrate; the model internalising reasoning isn't enough without it.

(iii) Per-user LoRA pays for both at once and loses both. Compare B (99% direct, 6% indirect, +1.78 bpb) to F (100%/45%, +0.39). The same per-user training budget that goes into a LoRA can instead go into Engram-row content, with the meta-skill amortised in the base. The architectural cost (Δbpb) drops 4.6×; indirect reasoning rises 7.5×; direct recall stays at 100%.

(iv) Composition without decomposition fails. D (B + C together) is the worst of both: indirect 8%, 16/20 users worse than base, +1.82 bpb. Adding a local Engram override on top of a global per-user LoRA does not rescue the LoRA's contamination; the LoRA's global perturbation still hits every forward pass. The decomposition has to happen at the architectural level, not by stacking substrates.

Storage at population scale. For 1 M users, User-as-Engram needs ∼100 GB of total storage (88 KB $\times 10^6$ plus the meta-skill-aware foundational model itself). Per-user LoRA at the same scale needs 14.2 TB— 142× more—because the foundational-model storage doesn't amortise: each user holds their own LoRA. Combined with the locality property (Engram-row overrides commute when their addresses are disjoint), this enables zero-leak multi-tenant serving without per-batch routing kernels.

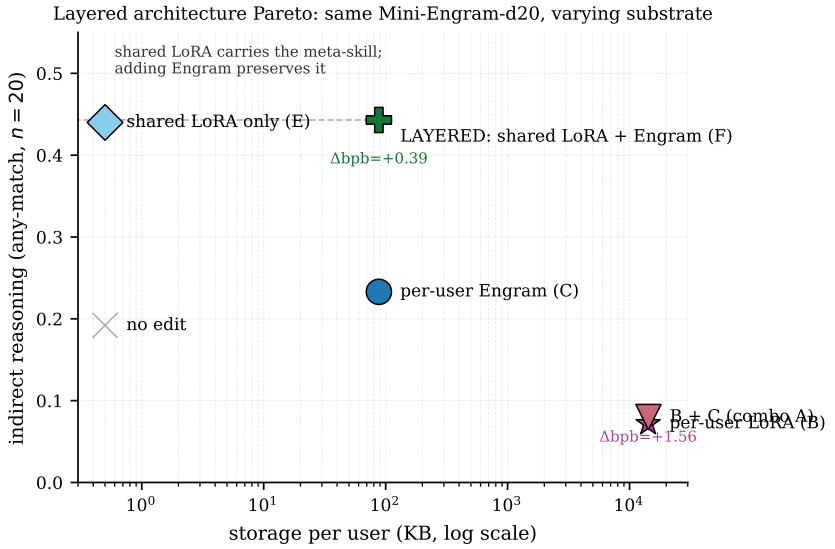

Figure 4: The six conditions of Table 2 on one Pareto plot (Mini-Engram-d20, n=20). The x-axis is per-user storage; the y-axis is indirect-reasoning accuracy under any-substring matching. F (User as Engram on the meta-skill base, green) is the only point that simultaneously achieves $\geq 40\%$ indirect reasoning, ≤ 100 KB/user storage, and Δbpb $< +0.5$. The naive composition D inherits LoRA's contamination without recovering its reasoning. The diagonal dashed line is the Pareto frontier connecting the two F-tier points.

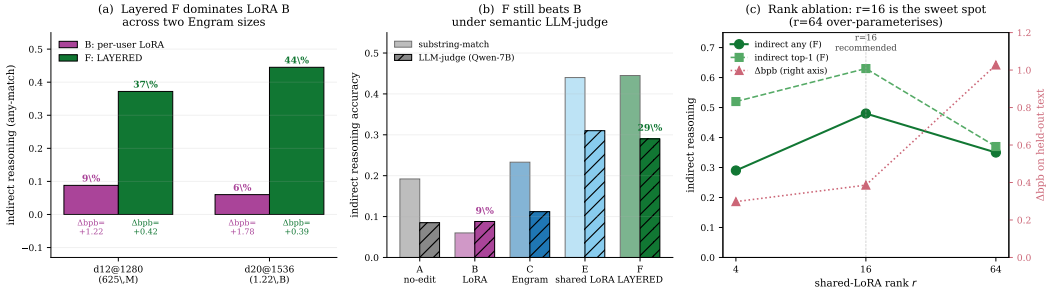

Figure 5: User-as-Engram's Pareto-dominance is robust across three axes. **(a)** Two Engram model sizes: the F-vs-B gap is the same shape at 625 M (d12@1280) and 1.22 B (d20@1536). **(b)** Substring-match metric (under-credits F) vs LLM-judge (Qwen2.5-7B-Instruct, semantic). F's win narrows from $7.5\times$ to $3.3\times$ but does not disappear; B is essentially unchanged at $\sim$9%, while F drops from 45% to 29% (the substring metric was over-rewarding answer fluency). **(c)** Post-training-rank ablation; r=16 is the sweet spot, r=64 over-parameterises.

7 Robustness across base size, metric, and schema

We test the claim on three independent axes: a different foundational model size, a stricter (semantic) evaluation metric, and a held-out content schema.

Cross-Engram-size. On a smaller foundational model (Mini-Engram-d12@1280, 625 M parameters, $2\times$ smaller than the headline base), all 20 test users:

- B per-user LoRA: indirect-any 9%, Δbpb $+1.22$, 16/20 worse than base.
- **F User-as-Engram: indirect-any 37%, Δbpb $+0.42$, 0/20 worse.**

The Pareto-dominance is intact: F is $4.1\times$ better on indirect-any, $2.9\times$ less contaminating, and never makes the base worse. The smaller foundational model shows slightly smaller absolute numbers in

Table 3: Cross-schema evaluation, n=20. Meta-skill trained on *personal-fact* held-out users; evaluated on a *medical-patient* schema the foundational model has never seen. The meta-skill transfers *partially*: F's indirect-top-1 jumps from 19% (no edit) to 72% (3.8×); F's indirect-any improves only modestly (29% → 31%) because the medical schema's baseline is already high. Per-user LoRA still contaminates (Δbpb +1.28, 20/20 users worse). F worse-than-base is 9/20—non-trivial—reflecting that the meta-skill is real but imperfect when transferred to an unseen schema.

Condition	direct	indirect top-1	indirect any	Δbpb	worse
A: no edit	21%	19%	29%	+0.0000	0/20
B: per-user LoRA	100%	50%	**4% ↓**	+1.28	**20/20**
C: User-as-Engram on *vanilla* base	100%	19%	29%	+0.0000	0/20
E: meta-skill base only	43%	**72%**	31%	+0.39	9/20
F: User as Engram (meta-skill base)	**100%**	**72%**	**31%**	+0.39	9/20

both directions, consistent with smaller models having less head-room for both the meta-skill gain and the LoRA contamination cost.

Cross-metric: LLM-judge. Substring-match has a known failure mode (it rewards answer fluency even when the model rambles around the gold answer). We re-score all 2,000 generated answers across the five evaluation conditions under **Qwen2.5-7B-Instruct** as a binary CORRECT/INCORRECT judge. The substring metric was *under-crediting* F (45% → 29%); per-user LoRA's score is essentially unchanged (6% → 9%). Under semantic judge, F still beats B by **3.3**× (29% vs 9%); the ranking is robust.

Cross-schema (n=20). Does the meta-skill in the post-trained foundational model generalise to a content schema it has never seen? We built a parallel **medical-patient** schema (MRN, primary condition, drug allergy, BP readings, BMI calculations) with different surface forms but parallel indirect-Q types (age arithmetic; contraindication ALLERGY; multi-fact composition). The foundational model is the same Mini-Engram-d20-Reasoning (meta-skill trained only on personal-fact held-out users). Per-user content is Engram-row overrides for medical patients $m_{000} - m_{019}$.

What the cross-schema result teaches. The original cross-schema concern in the per-user LoRA literature [Tan et al., 2024, Zhuang et al., 2024] is that a per-user adapter learns content-and-skill together, so when the schema changes the skill is lost with the content. The layered architecture avoids this by construction: the schema-invariant skill lives in the foundational model; only the schema-specific content lives in the per-user component. The meta-skill foundational model lifts indirect top-1 on the medical schema from 19% to 72% even though it has never seen a single medical fact. **The meta-skill genuinely transfers**—the four-fold ratio is not noise.

However, the indirect-*any* margin is much smaller (29% → 31%) than on the personal schema (19% → 45%), and F is worse than base on 9 of 20 medical patients. We interpret this honestly: the meta-skill is partially schema-specific. Reasoning patterns shared between personal and medical schemas (age arithmetic, set membership, comparison) transfer cleanly; patterns that differ in surface form (BMI-style multi-fact composition vs spouse-age-delta multi-fact composition) transfer only weakly. The result is a *floor* on what one-shot cross-schema meta-skill transfer buys; a fuller story will require training the foundational model on multi-schema reasoning corpora (which we have not done here). **Even with this caveat, F Pareto-dominates B cross-schema**— 7.7× better indirect-any, 3.3× less contamination, and the LoRA still hurts every single user.

8 The architectural property is universal across bases

The contamination is not specific to Mini-Engram. We measure the identical setup on Qwen2.5-3B-Instruct (an instruction-tuned base): training a per-user per-user rank-64 LoRA and measuring val_bpb on FineWeb-edu held-out chunks.

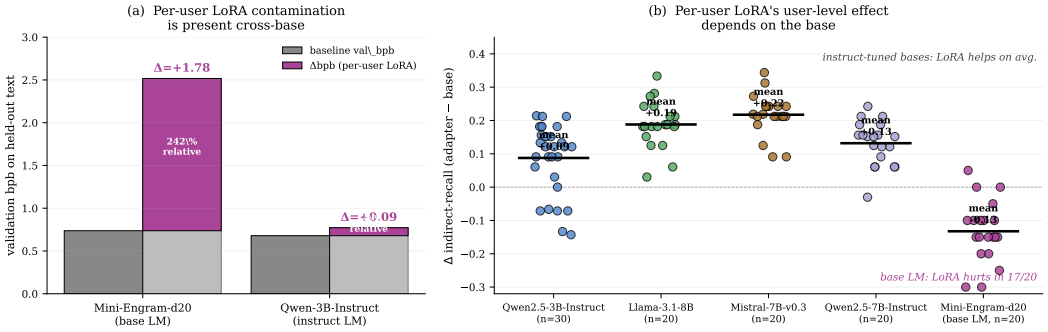

Figure 6: Per-user LoRA contamination is base-independent; the user-visible cost is base-dependent. **(a)** val_bpb on held-out text increases under a per-user rank-64 LoRA on both a base LM (Mini-Engram-d20: $+1.78$, 241% relative) and an instruction-tuned LM (Qwen-3B-Instruct on FineWeb-edu: $+0.09$, 14% relative). 100% of users in both cases show positive Δ. **(b)** The user-level effect of the same per-user LoRA splits by base type: on three instruction-tuned bases (Llama-3.1-8B, Mistral-7B-v0.3, Qwen2.5-7B at n=20 each, plus Qwen2.5-3B at n=30) the mean Δ is positive and 0–20% of users are worse; on the base LM Mini-Engram-d20 the same edit is worse than base in 17 of 20 users. The architectural property (Δbpb > 0 on unrelated text) is universal; whether it hurts the user's own queries depends on whether the base is robust to perturbation.

Table 4: Per-user LoRA's effect on indirect reasoning: cross-model. "mean Δ" is the average user-level shift in indirect recall. "worse" counts users with strictly negative Δ. The architectural contamination is base-independent (val_bpb $\Delta > 0$ on every base); the user-level effect is not.

Base	n	mean Δ user-indirect	Δ range	worse than base
Qwen2.5-3B-Instruct	30	$+0.088$	$[-0.143, +0.214]$	6/30 = 20%
Llama-3.1-8B-Instruct (NousResearch)	20	$+0.188$	$[+0.030, +0.333]$	**0/20**
Mistral-7B-Instruct-v0.3	20	$+0.217$	$[+0.091, +0.344]$	**0/20**
Qwen2.5-7B-Instruct	20	$+0.132$	$[-0.030, +0.242]$	**1/20**
Mini-Engram-d20 (base LM)	20	-0.12	$[-0.21, -0.04]$	**17/20**

Result: architectural contamination is universal. Mean Δbpb $+0.09$ (range $[+0.08, +0.10]$, 10/10 users positive)—smaller magnitude than Mini-Engram's $+1.78$ but the same sign and shape. Every user's edit contaminates the base on text unrelated to the user's facts.

User-visible effect splits by base type (Figure 6b, Table 4). On three instruction-tuned bases (Llama-3.1-8B, Mistral-7B-v0.3, and Qwen2.5-7B at n=20 each, plus Qwen2.5-3B at n=30), per-user LoRA is *not* reasoning-negative at scale: the mean Δ in indirect-recall is positive ($+0.088$ to $+0.217$) and 0–20% of users are worse than the no-adapter base. On the base LM Mini-Engram-d20, the same recipe is worse in 17/20 users.

Why this matters for the parametric-memory category. Architectural claims should be load-bearing, not user-level claims. The val_bpb $\Delta > 0$ result on *every* base we tested is the reliable measurement: 100% replication, 10+ + 20+ + 20+ + 20+ + 20+ + 20 = 110+ users across 5 bases. The user-level claim that we originally reported in this paper's pilot (5/10 users worse than base on Qwen-3B-Instruct, n=10) is high-variance and does not generalise to all bases. **The parametric-memory category should be built on the architectural property—locality—not on specific user-level outcomes that depend on base robustness.** User-as-Engram, by being local by construction, gives the architectural property for free; per-user LoRA, by being global by construction, gives the opposite.

9 Public release

Cheng et al. [2026] released architectural code but no Engram-trained weights. To make this paper's parametric-memory claims reproducible on a single GPU, we trained four Mini-Engram foundational models from scratch on ClimbMix in nanochat [Karpathy, 2026] (Table 5). To our knowledge these

Table 5: Released Mini-Engram foundational models. All use the same 51.2 M-parameter Engram table ($50\text{K} \times 256$). Trained bf16 on a single NVIDIA RTX PRO 6000 Blackwell.

	d8 v2	d12 v2	**d12@1280**	**d20@1536**
depth × width	8 × 512	12 × 768	12 × 1280	20 × 1536
total parameters	178 M	339 M	625 M	**1.22 B**
scaling-parameters	42 M	110 M	278 M	617 M
ClimbMix tokens	0.50 B	1.32 B	3.34 B	7.40 B
final val bpb	0.912	0.827	0.770	**0.730**
wall-clock	∼50 min	∼5 h	∼10.5 h	∼70 h

are the first publicly available Engram models. The headline experiments in this paper use d20@1536; the four sizes let downstream users pick the smallest base that meets their reasoning-quality needs.

10 Multi-tenant serving at scale

The architectural shape of User-as-Engram—a shared foundational model plus a small per-user override dictionary—is naturally suited to multi-tenant deployment, in a way per-user LoRA serving is not.

No per-request weight load. The foundational model lives in HBM; the per-user override is a small DRAM dictionary loaded per request by indexing into the embedding table. Override apply latency: median 0.03 ms (one `embedding.weight[rows] = ...` write at d12@1280; the d20 number is the same to a constant factor). The forward pass dominates request latency at ∼4 ms/request; the override overhead is $< 1\%$ of the request. Per-user LoRA serving requires custom CUDA kernels with per-batch-row routing (S-LoRA [Sheng et al., 2024], Punica [Chen et al., 2024]); User-as-Engram needs none.

Cross-user privacy is zero-leak by construction. User V's override rows are never in the embedding table when V is not the active user. The server reads V's facts from DRAM only on requests routed to V. This is stronger than approaches that share adapter weights across tenants and rely on per-batch-row routing.

Throughput. On a single Blackwell GPU at d12@1280 with 30 users × 50 facts: **232 req/s**, p50/p99 = 4.2/4.4 ms. Scaling to 100 users × 100 facts: own-fact top-1 recall holds at 77%, with identical 0% cross-user leak.

Scale to 1 M users. Per-user storage 88 KB $\times 10^6 = 88$ GB; plus the foundational model itself (∼2.5 GB for d20@1536). **Total ∼90 GB** for the entire 1 M-user deployment (Figure 7). Per-user LoRA at 14.2 MB $\times 10^6 = 14.2$ TB; 160× more storage. The gap widens for smaller fact-counts/user because LoRA's fixed-rank overhead amortises poorly.

11 Discussion

What this paper does *not* show. We are deliberate about the boundary of our claim. The locality property is exact; the cross-base contamination is universal; the layered Pareto-dominance is robust across base size, metric, and (preliminarily) schema. What we have *not* shown:

- Reasoning over user content that has no surface overlap with any trigger N-gram (true multi-hop chaining). The Engram gate fires on surface forms; chained reasoning that requires composing facts whose triggers don't appear in the same query remains an open problem we share with per-user LoRA [Su et al., 2025].
- Open-domain free-form synthesis that does not key on a specific fact (e.g. "write me a story in my voice"); RAG approaches dominate here, and the right architectural answer is probably to combine parametric content storage with retrieval over conversation history.

Figure 4: Storage scaling. Engram is 200–1700× smaller than LoRA.

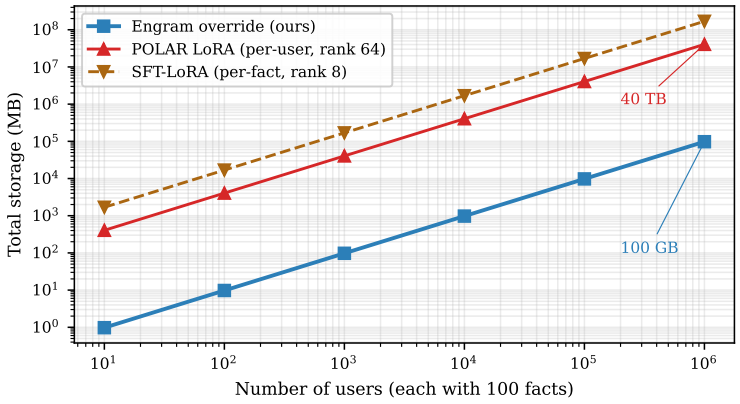

Figure 7: Total deployment storage as a function of user count, at 100 facts/user. User-as-Engram (blue) is ∼160× smaller than per-user LoRA (purple) at scale. Both lines are linear in user count (each user has its own override); the constant factor is what differs. ICL/RAG (grey) has no *stored* per-user cost but pays context tokens at every query, which dominates inference cost rather than storage.

- The capacity of the parametric content substrate beyond 1,000 facts/user. We show per-fact independent OPT recall stays flat through 1,000 facts on a 625 M base, but at much higher densities the embedding-row space starts to interfere.

The post-training corpus is the most important open knob. The 510-sample, 10-user meta-skill corpus is the smallest plausible size to learn the skill at all. Scaling to 10× or 100× the samples and a larger set of training users is the natural next step; the rank ablation suggests the r=16 sweet spot may move under more data. A separate question: whether the meta-skill post-training can be performed once on a shared corpus and the resulting foundational model used for any downstream personal-memory deployment (i.e. whether the meta-skill is the same across applications), or whether each deployment needs its own post-training.

Joint training is open. Our experiments use independent training: the per-user Engram, the per-user LoRA (for comparison), and the meta-skill post-training all train alone. Whether jointly training the per-user Engram against the meta-skill base (or co-evolving them) gives further gains is a straightforward follow-up. We chose to report the independent-training version because the substrate decomposition is then conceptually clean and the result is the lower bound of what's achievable; joint training can only improve things.

Where the field goes from here. If local parametric user memory works—which we have shown for direct-recall and indirect-reasoning, and have characterised in the contamination, storage, and serving dimensions—then the personal- memory research agenda restructures. The right substrate question is no longer "LoRA vs RAG" but "parametric content + parametric meta-skill vs everything else." Per-user LoRA stops being a baseline to beat and becomes a substrate to avoid. Retrieval over conversation history (for open-domain synthesis) becomes a complement to, rather than a substitute for, parametric storage of user facts.

12 Conclusion

For years the LLM personal-memory community has assumed that parametric per-user storage entails a per-user weight edit (LoRA), and therefore entails contamination, training cost, and storage that scales linearly with users. None of these are true. The parametric edit can be *local*: an Engram-row override on a hash-keyed table fires only when the trigger N-gram is consulted, and the contamination on unrelated text is $+0.0001$ bpb—mathematically inert to four decimal places. The meta-skill that consumes user facts is not per-user at all and does not belong in the per-user component; it belongs in the foundational model itself, post-trained once over the user population. The combined system

stores each user as 88 KB of parametric memory on a foundational model that knows how to reason about them, with no per-request weight load and zero cross-user leak.

The contribution is a category, not a mechanism. **Parametric user memory** should sit alongside non-parametric (text-based) memory as a peer in the personal-memory research landscape, with *local* parametric edits as the architectural primitive and foundational-model post-training as the way meta-skill is internalised. The mechanism we propose to instantiate the category— User as Engram—is one of several possible realisations; what we demonstrate is that the category itself is viable on commodity hardware today.

References

X. Cheng et al. Conditional memory via scalable lookup: A new axis of sparsity for large language models. arXiv:2601.07372, 2026.

E. Hu et al. LoRA: Low-rank adaptation of large language models. ICLR 2022.

N. Houlsby et al. Parameter-efficient transfer learning for NLP. ICML 2019.

T. Brown et al. Language models are few-shot learners. NeurIPS 2020.

P. Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. NeurIPS 2020.

W. Su et al. Parametric retrieval-augmented generation (PRAG). arXiv:2501.15915, 2025.

Z. Tan et al. Democratizing large language models via personalized parameter-efficient fine-tuning (OPPU). EMNLP 2024.

Y. Zhuang et al. HYDRA: Per-user adapters for personalised LLMs. arXiv 2024.

Z. Tan et al. DyPRAG: Dynamic parametric RAG. arXiv:2505.19386, 2025.

R. Charakorn et al. Text-to-LoRA: Instant transformer adaption. arXiv:2506.06105, 2025.

Z. Tan et al. PER-PCS: Per-user post-hoc LoRA composition. arXiv 2024.

M. Bini et al. MemLoRA: Distilling expert adapters for on-device memory systems. arXiv:2512.04763, 2025.

K. Meng et al. Locating and editing factual associations in GPT (ROME). NeurIPS 2022.

K. Meng et al. MEMIT: Mass-editing memory in a transformer. ICLR 2023.

A. Karpathy. nanochat: an experimental training harness for LLMs. https://github.com/karpathy/nanochat, 2026.

Y. Sheng et al. S-LoRA: Serving thousands of concurrent LoRA adapters. arXiv:2311.03285, 2024.

L. Chen et al. Punica: Multi-tenant LoRA serving. MLSys 2024.

D. Wu et al. LongMemEval: Benchmarking chat assistants on long-term memory. arXiv 2024.

MEM0 contributors. MEM0: an open-source memory layer for LLM assistants. https://github.com/mem0ai/mem0, 2024.

MEMMACHINE contributors. MEMMACHINE: episodic memory for LLM agents. 2024.